Scientific Forgery Image Detection

By: Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam

May 8, 2026

Software Engineering Tools, CS 3338

Table of Contents

1. Introduction

1.1 Purpose

Scientific images play an important role in academic research and published studies. However, some images may be manipulated through techniques such as copy-move forgery, where parts of an image are duplicated and placed elsewhere to falsify results.

The purpose of this system is to detect and analyze copy-move forgery in biomedical and scientific images. Users will be able to upload images through a web interface, and the system will process them to identify potentially duplicated regions. This helps support scientific integrity by assisting researchers and reviewers in identifying manipulated figures.

1.2 Intended Audience

This document is intended for:

- Software developers working on the system
- Course instructors evaluating the project
- Students involved in system design
- Future maintainers of the software

1.3 Overview

The system is a complete web-based application consisting of a frontend interface, backend API, and integrated detection module. The full system is containerized using Docker.

At this stage, the system includes:

- Complete frontend image upload interface with image preview
- Backend API with image validation and error handling
- OpenCV-based copy-move forgery detection
- Refined results display with marked suspicious regions
- Containerized deployment using Docker Compose

2. System Architecture

2.1 Overview

The system follows a simple client-server architecture.

2.2 Frontend

The frontend is responsible for user interaction.

Responsibilities:

- Provide image upload interface
- Send image data to backend
- Display backend responses
- Show suspicious regions

Technologies:

- React.js
- HTML/CSS

2.3 Backend

The backend handles incoming requests from the frontend.

Responsibilities:

- Receive uploaded images
- Validate image file format
- Run fraudulent region detection
- Return placeholder or future detection results

Technologies:

- Python
- FastAPI
- Uvicorn
- OpenCV
- Numpy
- Pillow

2.4 Detection Module

The detection module analyzes uploaded images for copy-move forgery.

Responsibilities:

- Detect keypoints in images using ORB
- Match keypoints to identify duplicated regions
- Highlight suspicious regions on the output image
- Return detection results to backend

2.5 Deployment

The system is containerized using Docker for consistent deployment.

Services:

- Frontend container (nginx serving HTML)
- Backend container (FastAPI + Uvicorn)
- Database container (PostgreSQL — planned for future use)

2.6 Workflow Design

Workflow:

1. User uploads biomedical image through frontend interface
2. Frontend shows image preview and sends to backend via POST request
3. Backend validates image file format
4. If invalid, backend returns error message

5. If valid, detection module analyzes image for duplicated regions
6. Suspicious regions identified and marked on output image
7. JSON response returned to frontend with results
8. Frontend displays marked image and status message

3. User Interface Design

3.1 Homepage

The homepage introduces the application and provides navigation to upload functionality.

Elements:

- Project title
- Short description
- Navigation buttons

3.2 Image Upload Interface

This page allows users to submit images for analysis.

Elements:

- Upload button
- File selection option
- Image preview
- Submit button

3.3 Results Display

- Confirmation of upload
- Detection status message
- Marked image showing highlighted suspicious regions
- Match count indicator

4. Glossary

Term	Definition
API	Application Programming Interface used for communication between frontend and backend
Backend	Server-side system that processes requests
Frontend	User interface of the application
Copy-Move Forgery	Image manipulation where parts of an image are duplicated within the same image
React.js	JavaScript library used for building user interfaces
FastAPI	Python framework used for building APIs
Uvicorn	Server used to run FastAPI applications
OpenCV	Open-source computer vision library used for image analysis
ORB	Oriented FAST and Rotated BRIEF — keypoint detection algorithm
Keypoint	A distinctive location in an image used for matching
Docker	Platform for containerizing applications
Docker Compose	Tool for defining multi-container Docker applications
REST	Representational State Transfer — a standard for web API communication

5. Versions Table

Version	Date	Description	Authors
1.0	May 8, 2026	Initial Software Design Document created for Snapshot 1	Janelle Rivera, Eric Marroquin, Alexis Flores

1.1	May 10, 2026	Updated frontend implementation progress and UI design	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.2	May 12, 2026	Added fraudulent region detection workflow and frontend/backend integration updates	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.3	May 14, 2026	Added final workflow refinements, result display improvements, and finalized system updates	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam

6. References

- FastAPI Documentation: https://fastapi.tiangolo.com/
- React Documentation: https://react.dev/
- OpenCV Documentation (future reference): https://opencv.org/